DATA PREPROCESSING COURSE

Scikit-learn Pipelines

From raw data to a trained model—and reliable predictions



RAW DATA
↓
PREPROCESS
↓
LEARN
↓
PREDICT

LECTURE • CONCEPTS + END-TO-END PYTHON WORKFLOW

Learning goals

By the end of this lecture, you should be able to build a complete supervised-learning workflow.

- Separate features (X) from the target (y).
- Split data correctly into training and test sets.
- Handle missing values, categorical columns, and numeric scales.
- Combine preprocessing and a model into one Pipeline.
- Fit, evaluate, predict, and reuse the trained workflow.
- Recognize data leakage and common implementation mistakes.

Core idea

A pipeline makes preprocessing and learning one reproducible object.

Example task

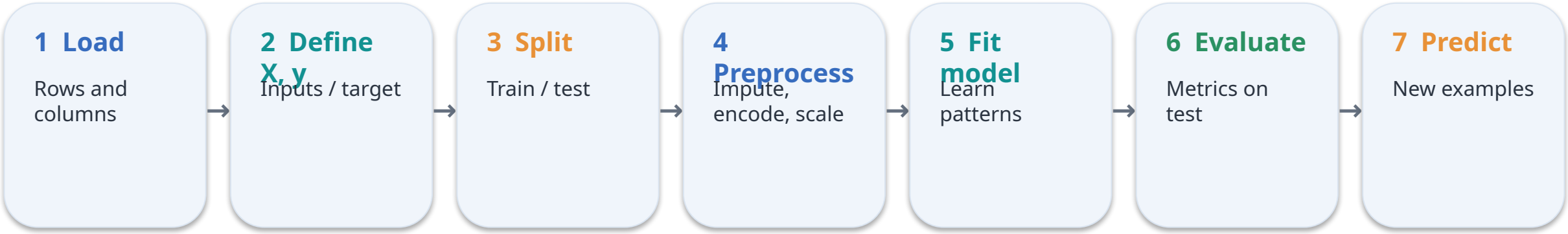
Predict a numeric target such as house price from mixed tabular features.

Assumption

We have labeled historical examples: each row has input features and a known target.

The full supervised-learning workflow

The model is only one step; the data preparation must be learned from training data too.



Important: split first. Fit preprocessing only on the training partition; then apply the learned transformations to validation, test, and future data.

What is a scikit-learn Pipeline?

A sequence of transformations followed by a final estimator.

```
    from sklearn.pipeline import Pipeline
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import
LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", LogisticRegression())
])

pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)
```

- Each step has a name and an object.
- Intermediate steps transform X.
- The final step is usually the estimator/model.
- fit() learns every step in order.
- predict() applies the same learned steps, then predicts.

Why it matters

Fewer disconnected scripts, fewer train/test inconsistencies, and easier cross-validation or deployment.

Start with the data: X and y

Most scikit-learn estimators expect a feature matrix X and target vector y.

```
import pandas as pd

df = pd.read_csv("customers.csv")

target = "will_churn"
X = df.drop(columns=[target])
y = df[target]

print(X.shape, y.shape)
print(X.dtypes)
print(y.value_counts())
```

X = features

Predictor columns: age, plan, monthly spend, etc.

y = target

The label we want to predict: class, score, price, outcome.

- Remove identifiers or columns that would not be available at prediction time.
- Check target quality, data types, duplicates, impossible values, and class balance.

Split before learning preprocessing

The test set is a final, untouched estimate of how the whole workflow generalizes.

```
from sklearn.model_selection import
train_test_split

X_train, X_test, y_train, y_test =
train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y  # classification; omit for
regression
)
```

- Train set: fit imputers, encoders, scalers, and model.
- Test set: evaluate after decisions are complete.
- random_state: reproducible split.
- stratify=y: preserves class proportions for classification.
- For time-dependent data, use a time-aware split instead of random shuffling.

Avoid leakage

Do not impute, scale, select features, or tune using the full dataset before splitting.

Preprocessing depends on column type

A common tabular dataset contains both numeric and categorical columns.

Numeric columns

Examples: age, income, measurements

Missing values → impute

Scale when useful → standardize

Categorical columns

Examples: city, plan, product type

Missing values → impute

Text categories → one-hot encode

Target y

Classification: class labels

Regression: numeric value

Do not encode the target as if it were an input feature.

Column-specific processing is handled with ColumnTransformer, then combined with the model in a single Pipeline.

```
raw X  →  ColumnTransformer (numeric + categorical)  →  estimator  →  prediction
```

Step 1 — Handle missing values

Imputation learns a replacement value from training data.

```
    from sklearn.impute import SimpleImputer

numeric_imputer = SimpleImputer(
    strategy="median"
)

categorical_imputer = SimpleImputer(
    strategy="most_frequent"
)
```

- Median is robust to skew and outliers.
- Mean can be suitable for roughly symmetric numeric data.
- Most-frequent fills with the common category.
- Constant can add an explicit value such as 'Unknown'.
- Fit on training data only; reuse those learned values elsewhere.

Note

Missingness may carry information. In some tasks, add a missingness indicator and compare performance.

Step 2 — Encode categorical features

Most estimators need numeric feature representations.

```
 from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    handle_unknown="ignore"
)

# Example:
# plan = ["basic", "premium", "basic"]
# becomes indicator columns such as
# plan_basic, plan_premium
```

- One-hot encoding creates indicator columns for categories.
- handle_unknown='ignore' avoids an error when future data contains an unseen category.
- Do not assign arbitrary integers to nominal categories unless their ordering is meaningful.
- High-cardinality columns can create many features; consider domain-aware alternatives.

Key rule

The encoder learns the category vocabulary during fit().

Step 3 — Scale numeric features when useful

Scaling puts numeric features on comparable ranges.

```
from sklearn.preprocessing import
            StandardScaler

scaler = StandardScaler()

# Learns training mean and standard
deviation
# then transforms each feature:
# z = (x - mean) / standard_deviation
```

- Often useful: logistic regression, SVM, k-nearest neighbors, PCA, regularized linear models.
- Usually not necessary for decision trees and random forests.
- Scaling does not make a feature normally distributed.
- Outliers can affect mean and standard deviation; RobustScaler may be worth testing.

Leakage reminder

Never call fit_transform() on the entire dataset before splitting.

Run different transformations on different columns

ColumnTransformer applies each transformer to selected columns and joins the results.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_features = X.select_dtypes(include="number").columns
categorical_features = X.select_dtypes(
    exclude="number"
).columns

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_features),
    ("cat", categorical_pipe, categorical_features),
])
```

- Numeric pipeline: impute → scale.
- Categorical pipeline: impute → encode.
- ColumnTransformer routes columns to the right branch.
- Each branch can contain its own Pipeline.
- Explicit column lists are often safer than inferring types in production.

Build the complete classification pipeline

BUILD

This is the central pattern: preprocessing + model are fitted together.

```
        from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

clf = Pipeline([
    ("preprocess", preprocessor),
    ("model", LogisticRegression(max_iter=1000)),
])

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]  # binary
classification
```

One object, one workflow

- fit() learns imputation values, categories, scaling statistics, and model parameters.
- predict() transforms X_test using those learned steps.
- predict_proba() gives estimated class probabilities when supported.
- Use a different estimator or settings for multiclass tasks as appropriate.

The fitted cl

For regression, swap the final estimator

The preprocessing pattern is largely the same; the target and evaluation metrics change.

```
  from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline

reg = Pipeline([
    ("preprocess", preprocessor),
    ("model", RandomForestRegressor(
        n_estimators=300,
        random_state=42
    )),
])

reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)
```

- Classification predicts classes or class probabilities.
- Regression predicts a continuous number.
- Tree models usually do not need feature scaling.
- Choose the estimator based on task, data size, interpretability, and validation results.

Do not mix metrics

Accuracy is for classification; MAE/RMSE/R^2 are common regression metrics.

Evaluate the model on held-out data

Choose metrics that match the problem and the cost of mistakes.

```
from sklearn.metrics import (
    accuracy_score, classification_report,
    ConfusionMatrixDisplay, roc_auc_score
)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

# For binary classification when probabilities exist:
print("ROC AUC:", roc_auc_score(y_test, y_prob))

ConfusionMatrixDisplay.from_predictions(y_test,
y_pred)
```

- Accuracy: fraction correctly classified; can mislead with imbalanced classes.
- Precision: of predicted positives, how many were correct?
- Recall: of actual positives, how many were found?
- F1: harmonic mean of precision and recall.
- ROC AUC: ranking quality across thresholds; consider PR AUC for rare positives.

Thresholds matter

The default

Evaluate regression predictions

Look at error size and whether the model beats a simple baseline.

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test,
y_pred))
r2 = r2_score(y_test, y_pred)
```

- MAE: average absolute error in target units.
- RMSE: penalizes large errors more strongly.
- R^2: relative fit versus predicting the test-set mean; can be negative.
- Inspect residuals and errors across important subgroups.
- Always compare with a simple baseline.

Interpretability

A metric is not a complete diagnosis: investigate where and why the model fails.

Use cross-validation for a more stable estimate

Cross-validation evaluates a workflow across multiple train/validation splits.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    clf, X_train, y_train,
    cv=5,
    scoring="accuracy"
)

print(scores)
print("Mean CV accuracy:", scores.mean())
```

- Pass the entire Pipeline to cross_val_score.
- Each fold fits its own preprocessing only on that fold's training data.
- Choose a splitter appropriate to the data: stratified, grouped, or time-aware.
- Use CV on training data for model choices; keep test data for final evaluation.

Why Pipeline helps

It prevents preprocessing from leaking validation-fold information.

Tune model and preprocessing together

GridSearchCV can search named parameters inside a Pipeline.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "model__C": [0.1, 1, 10],
    "preprocess__num__imputer__strategy": ["mean", "median"],
}

search = GridSearchCV(
    clf,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
search.fit(X_train, y_train)

print(search.best_params_)
best_model = search.best_estimator_
```

- Double underscores navigate nested steps and parameters.
- Parameter names must match the actual pipeline structure.
- Use a scoring metric aligned with the task.
- RandomizedSearchCV can be more efficient for large search spaces.
- Do not repeatedly optimize against the test set.

Predict on new rows and save the fitted pipeline

DEPLOY

At inference time, provide the same input columns expected by the workflow.

```
      # New examples as a DataFrame with matching columns
new_customers = pd.DataFrame([
    {"age": 42, "monthly_spend": 85.0, "plan": "premium"},
    {"age": 27, "monthly_spend": 25.0, "plan": "basic"},
])

predictions = best_model.predict(new_customers)

import joblib
joblib.dump(best_model, "churn_pipeline.joblib")

# Later, in a compatible environment:
loaded_model = joblib.load("churn_pipeline.joblib")
new_predictions = loaded_model.predict(new_customers)
```

- Save the complete fitted pipeline, not only the estimator.
- Keep feature names, order, and types consistent.
- Track library versions and training-data provenance.
- Only load model files from trusted sources.

Operational habit

Test the save

Common mistakes to avoid

Most workflow bugs are about what information each step is allowed to learn.

- Preprocessing the full dataset before train/test split.
- Using test results repeatedly to choose features or hyperparameters.
- Encoding or scaling y as if it were a feature.
- Using random split for time-series or grouped observations.
- Leaving out handle_unknown for categories that may change.
- Evaluating only accuracy on a heavily imbalanced target.

- Training-serving skew: training and inference transformations differ.
- Target leakage: a feature indirectly contains the answer.
- Wrong metric: score does not match the business goal.
- Silent column drift: input schema changes after deployment.
- Reproducibility gaps: no fixed seeds, versions, or saved settings.

End-to-end template to adapt

REFERENCE

A compact reference implementation for a mixed-type classification dataset.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

df = pd.read_csv("data.csv")
X = df.drop(columns=["target"])
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
num = X.select_dtypes(include="number").columns
cat = X.select_dtypes(exclude="number").columns
prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), num),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("encode", OneHotEncoder(handle_unknown="ignore"))]), cat),
])
model = Pipeline([("preprocess", prep),
                  ("model", LogisticRegression(max_iter=1000))])
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print(classification_report(y_test, pred))
```

Takeaways

A reliable machine-learning workflow is a data workflow first.

1. Split early

Separate training and test data before learning preprocessing.

2. Preprocess by type

Impute, encode, and scale only where appropriate.

3. Chain it

Use ColumnTransformer inside Pipeline, with the estimator at the end.

4. Evaluate honestly

Use metrics and validation strategies suited to the task.

5. Reuse the workflow

Predict and persist the complete fitted pipeline.

6. Prevent leakage

Fit transformations only on the appropriate training fold.

A Pipeline is not just convenient syntax—it helps make the entire learning process reproducible and safer.

Quick check / discussion

ACTIVITY

Use these prompts to check understanding or run as a short in-class exercise.

- Why should the train/test split happen before imputation or scaling?
- Which steps would you use for numeric and categorical columns?
- What can go wrong if a new category appears after training?
- Would you scale features for a RandomForestClassifier? Why or why not?
- Which metric would you prioritize if missing a positive case is costly?
- How does passing a Pipeline to cross-validation prevent leakage?

Suggested exercise: build a pipeline for a small CSV, evaluate it, then predict two new rows.